Seamline artefact detection on CRAIG orthophotos

seam_qa: method, SeamNet architecture, dataset and synthetic data, and first results on the Eure-et-Loir nord 2025 sample · 20 October 2026

Summary

Mosaicking errors (bâtiment coupé, véhicule coupé, décalage, erreur de mosaïquage) happen *on* the seamline, where two source photos meet. seam_qa samples every seamline into **stations** every 4 m and scores each one by comparing the two photos at that point. The operator then reviews the highest-scoring stations in QGIS.

Result. In spatially grouped cross-validation, SeamNet finds **86%** of the on-seam artefacts when the operator reviews 5 candidates per km of seam. Gradient boosting on hand-made features finds 69% and random ranking 13%. At 20 candidates per km, SeamNet finds 100%.

Caveat. These are 29 artefacts from one campaign, in tiles centred on artefacts. The numbers show the approach works. They are not yet a production estimate.

1. The problem and the data

CRAIG's contrôle visuel (see *tuto_cv.pdf*) is done by hand: operators place a point on each artefact, and SG decides which ones are sent back to the vendor. The goal is to automate finding, locating and classifying the errors caused by mosaicking. The first deliverable is a ranked review list; the long-term target is fully automatic flagging.

Item	Content
Ortho tiles	200 m × 200 m GeoTIFF, RGBA, ~5 cm, EPSG:2154; one tile per artefact, named by art_id. 44 tiles here; ~70 GB over 3 campaigns in total.
Seamline graph	GrapheMosaiquage_EureLoir gpkg: 31,902 polygons in EPSG:3948 (CC48), one per source photo. Image = strip_photo (170 flight strips).
Artefact points	cv_artefact: 1,253 points (vallées Cher-Loir-Loire 1,027, Indre sud 168, Eure-et-Loir nord 58), all sent back (art_renvoi = 1).
BD TOPO	Buildings (with hauteur) and road sections, fetched per tile from the IGN WFS.

Two measurements shape the whole design. First, **seam errors sit on the seam** (Figure 1): every véhicule coupé point is within 2.2 m of a seamline, and most bâtiment coupé points within 3 m. Hotspots, cloud shadows and the unnamed types 3, 5, 15 and 21 sit 5–80 m away. They are a different problem and are left out of the seam model.

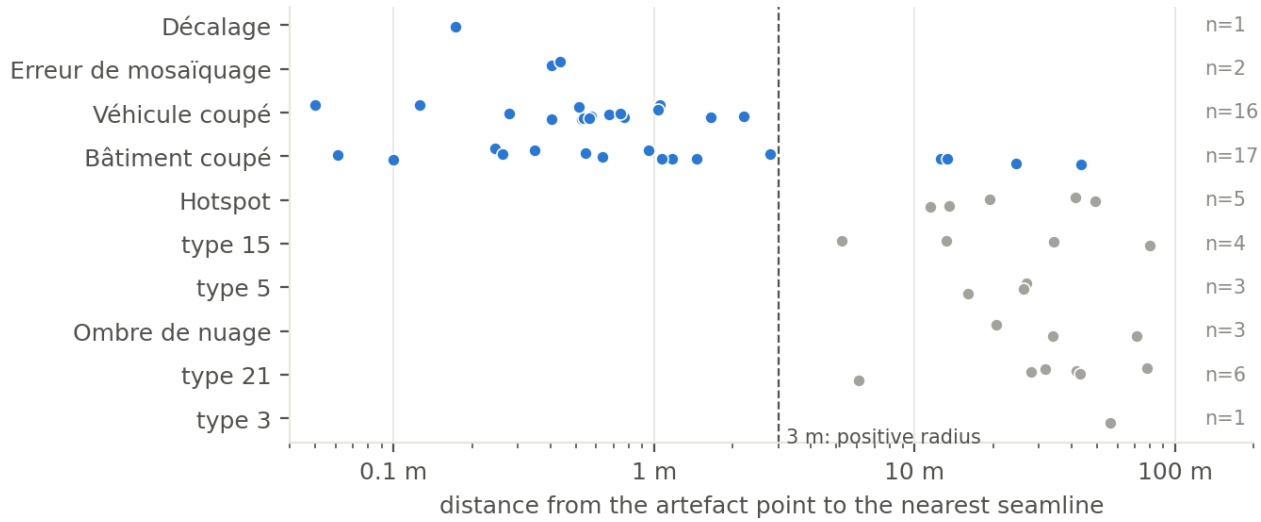


Figure 1. Distance from each Eure-et-Loir artefact point to the nearest seamline. Blue: seam-caused types used as targets. Grey: other types. Log scale.

Second, **the imbalance is extreme**. The Eure-et-Loir mosaic holds about 29,700 km of seams for about 38 seam artefacts, roughly one per 800 km. Inside the tiles it is better: about 466 m of seam per tile and usually one artefact. Here that gives 4,790 stations (19 km) and 42 positive stations, about 1 : 110.

2. Why the earlier attempts struggled

The earlier models (~/Downloads/CRAIG) were a YOLO hotspot detector and a DeepLabV3+ segmenter on 256 px good/bad patches along the seamlines. Neither has recorded metrics; the report calls them "pas encore optimales". The table lists the likely causes and what seam_qa changes.

Earlier approach	What changes in seam_qa
The network does not know where the seam is, nor which side is which photo; patches have arbitrary orientation.	Crops are rotated so the seam is vertical, plus a 4th channel holding the signed distance to the seam.
One 256 px scale.	Two scales: 5 cm for detail (vehicles), 10 cm for context (buildings).
Pixel masks needing near-millimetre precision.	One label per station (within 3 m of a point), with an ignore band around each point.
Every patch along every line, most of them trivially clean.	Context features (BD TOPO, parallax) feed the ranking; synthetic positives add variety.
Few positives.	synth.py builds artefacts on clean seams, always on one side only.
No honest metric.	Grouped CV by area, point matching, recall at a review budget (detections per km of seam).

3. Pipeline overview

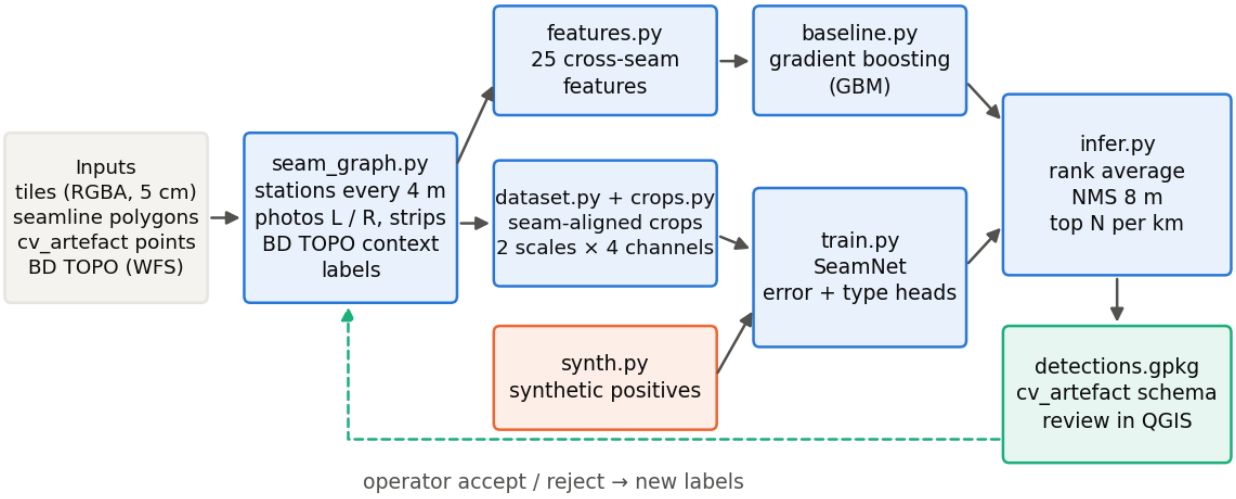


Figure 2. The seam_qa pipeline. Blue: scripts. Orange: synthetic data. Green: the QGIS review layer, whose accept/reject decisions can become new labels.

Script	Role
seam_graph.py	Seam lines, stations, photos on each side, BD TOPO context, labels; mosaic.vrt
features.py	25 cross-seam image features per station
baseline.py	Rule and gradient-boosting baselines, grouped CV, gbm_all.joblib
crops.py / dataset.py	Seam-aligned crop reader; two-scale crop cache
synth.py	Synthetic seam artefacts (4 types)
train.py	SeamNet: grouped CV, then the final model seamnet.pt
infer.py	Rank-average scores, NMS, top N per km, export detections.gpkg
metrics.py	NMS, point matching, recall at budget, AP

4. Seam graph and stations (seam_graph.py)

Seam lines. Each photo polygon's boundary is shared exactly with its neighbours: in a test tile, the summed boundary length is exactly twice the merged length. The boundaries of all polygons touching a tile are therefore noded, merged into lines (shapely *unary_union* + *linemerge*) and clipped to the tile.

Stations. Every 4 m along each line, a station stores its position and the seam tangent, estimated over ±3 m. Points 1.5 m to the left and right of the line are tested against the polygons to find the photo on each side (img_L, img_R) and their flight strips. A station whose two sides show the same photo, or no photo, is dropped. Stations from overlapping tiles closer than 2 m are deduplicated.

Context. From BD TOPO, per station:

- whether the seam crosses a building within ±2 m, and that building's height;
- distance to the nearest building;
- margin to the nearest road edge (centreline distance minus half the carriageway width);
- whether the two photos come from different strips.

A physical prior completes the context. A building of height h is displaced differently in the two photos. Approximating each photo's nadir by its polygon centroid, the mismatch is about $h \times baseline / H$, where the baseline is the distance between the two nadirs and $H \approx 1{,}250$ m is the flying height. Expressed in pixels, this **parallax_px** predicts how badly a building is cut.

Label	Rule	Stations
positive (1)	within 3 m of a seam-type point (types 2, 8, 13, 16)	42
ignored (−1)	3–30 m from a seam-type point, or within 10 m of any other point	585
negative (0)	everything else in the tiles (weak: the operator looked here and flagged nothing)	4163



Figure 3. Tile 3788 (véhicule coupé). Left: seam lines (white), stations and their labels; the yellow stars are the two artefact points, merged into one instance. Right: SeamNet's out-of-fold score for each station.

5. Seam-aligned crops (crops.py, dataset.py)

For every station, a square crop is resampled from a VRT of all the campaign's tiles. The crop is rotated so that walking up the crop follows the seam tangent, with the left photo on the left. The mapping from crop pixel to source pixel is a pure rotation, applied in a single *cv2.warpAffine* after a window read at the output resolution.

The seam lines near the station are drawn into the crop. The two regions on either side of the station (connected components) give the **side map**: +1 for the left photo, −1 for the right, 0 for any third photo. Combined with the distance transform, this gives the 4th input channel: side × min(distance, 32 px) / 32.

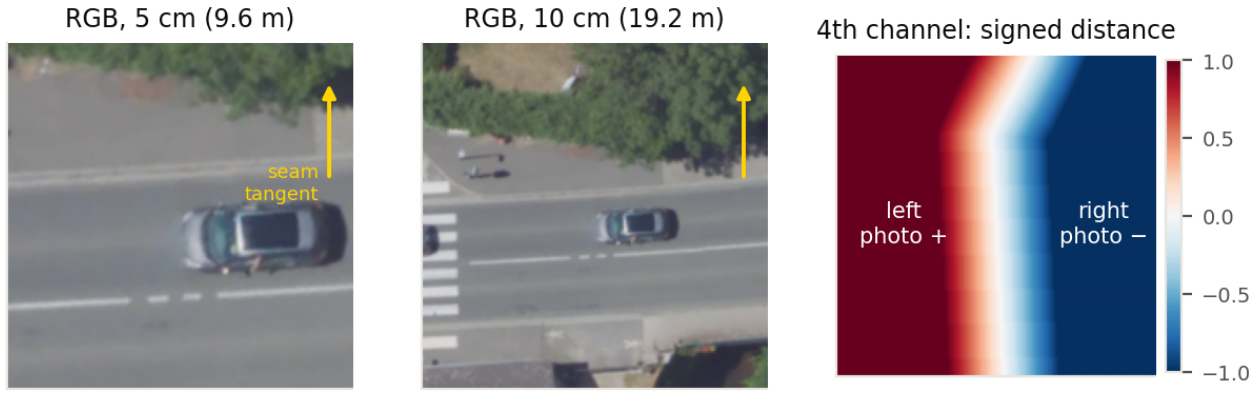


Figure 4. One station (tile 3766, véhicule coupé). The same point at 5 cm and 10 cm, and the signed-distance channel. The seam need not be straight; the channel follows the real line.

dataset.py caches both scales for every station (192 × 192 × 5 bytes per scale: RGB, side, distance), which takes 46 s and 1.8 GB here. For full campaigns, crops are read on the fly instead (about 33 ms per crop).

6. Hand-made features and gradient boosting (features.py, baseline.py)

The interpretable baseline uses a 256 px, 5 cm crop. In every crop row it finds the seam column (the left-to-right side change) and compares pixels just beside it:

Group	Features	Targets
Profile match	Correlation and median-corrected mean absolute difference between the grey profiles 3–9 px left and right of the seam (whole crop, and central half)	cuts, offsets
Shift search	Correlation of the along-seam gradient profiles at lag 0 vs the best lag (±2 m), the gain, and |lag|	décalage
Seam edge	Gradient across the seam vs 8–12 px beside it (mean ratio, p90)	visible cut
Colour	ΔE in Lab and ΔL between 2 m bands on each side; std ratio	raccord
Objects	98th-percentile saturation and 99th-percentile brightness, per side and their difference	vehicles
Context	bldg_cross, bldg_h, bldg_dist, road_margin, on_road, inter_strip, baseline_m, parallax_px, ...	prior

Model: sklearn HistGradientBoostingClassifier (depth 3, 150 iterations, learning rate 0.05, min 20 samples per leaf, balanced class weights), trained on labelled stations only.

7. Synthetic artefacts (synth.py)

With about 40 real positives per campaign, the network would mostly memorise them. synth.py creates positives from **clean** training stations, applying one rule: the change touches **one side of the seam only**. That is the definition of a mosaicking error: the two photos disagree at the seam. Every artefact's geometry is drawn in metres and applied identically to both scales.

Generator	What it does	Parameters	Mimics
object_cut	A blob straddling the seam is displaced on one side only	half-axes 2–8 m, shift 0.3–1.5 m, random direction	bâtiment coupé
vehicle_cut	A car exists on one side only (it moved between exposures)	1.6–2.0 × 3.8–5.0 m, 7 colours, windows, soft shadow	véhicule coupé
shift	The whole side is offset along/across the seam	along ±0.15–2 m, across ±0.3 m	décalage, mosaïquage
radiometric	Gain/offset jump between the two sides	gain 0.95–1.05, one channel ±6–10 %, offset ±6	raccord radiométrique



Figure 5. One example of each generator: the clean crop, then the synthetic artefact at both scales. The dotted line is the crop centre. Like real ones, some synthetic artefacts are hard to see.

This generalises the cut-shift augmentation from ~/Downloads/CRAIG/patch_tif_mosaic.ipynb, which drew random curved lines inside a patch, to the real seam of every crop. Generator weights are 0.30 / 0.30 / 0.25 / 0.15.

8. SeamNet

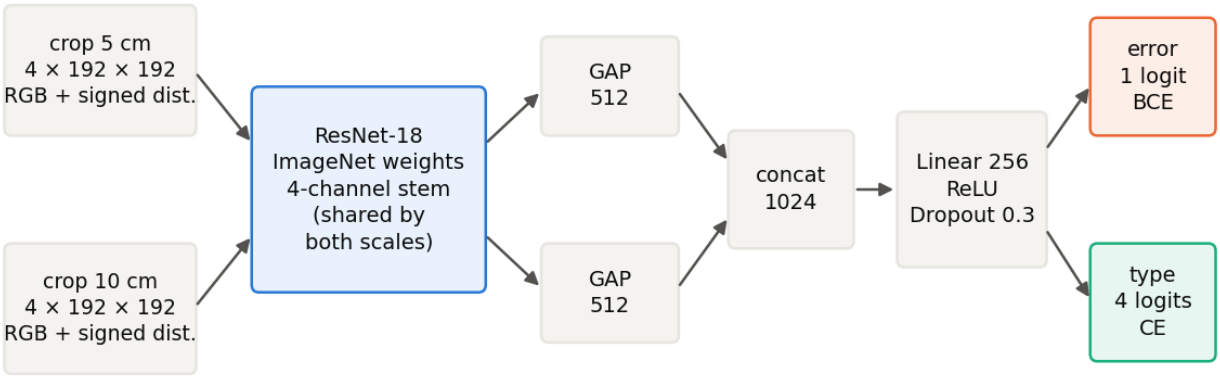


Figure 6. SeamNet (11.4 M parameters). One backbone sees both scales; their pooled features are concatenated before the two heads.

Architecture. A ResNet-18 (timm, ImageNet weights) with a 4-channel stem: timm adapts the pretrained RGB filters to 4 inputs. It runs on the 5 cm and 10 cm crops *with shared weights*, so the same filters learn "what a discontinuity looks like" at two zoom levels without doubling the parameters. Global average pooling gives 512 features per scale; the concatenated 1,024 go through Linear(256), ReLU and Dropout(0.3) into two heads:

- **error**: one logit, "is there a seam error at this station", trained with binary cross-entropy. This is the ranking score.
- **type**: four logits (bâtiment coupé, véhicule coupé, décalage / mosaïquage, radiométrie), trained with cross-entropy on positives only (weight 0.5). Real labels map types 2→0, 8→1, 13 and 16→2; synthetic ones take their generator's class.

Why this design. Telling the network where the seam is and which photo is on which side turns the task into a *comparison*: do the two sides agree? That is far easier to learn from a few examples than finding an artefact anywhere in an arbitrary patch. The task is framed as classification of a station, not segmentation, because the labels are points, not outlines.

Training recipe.

Setting	Value
Batch (32)	8 real positives + 8 synthetic positives (from training negatives) + 16 negatives
Augmentation	Flip along the seam; flip across it (which swaps the side code); brightness and contrast jitter
Optimiser	AdamW, weight decay 1e-4, one-cycle schedule up to 3e-4
Length	250 steps per fold (CPU, about 7–8 min per fold here); the final model trains on all stations
Inference	Probabilities averaged over the identity and an along-seam flip

9. Evaluation protocol

- **Spatially grouped CV.** Tiles within 1.5 km of each other form one group (27 groups), split into 5 folds. Overlapping tiles such as 339/340 can never land on both sides. Every score in Section 10 is out-of-fold.
- **Artefact instances.** Same-type points within 20 m are merged (339/340, 3559/3560, 3788/3789, …). This leaves 34 seam-type instances in the tiles, 29 of them within 5 m of a seam ("on-seam").
- **Detections.** Station scores go through non-maximum suppression with an 8 m radius. A detection finds an instance if it lies within 10 m of one of its points.

- **Review budget.** The top N detections, with N = budget × km of seam. Recall at 5/km means: the operator reviews 5 candidates per km of seam, and this share of the artefacts is among them.
- **AP.** Instance-level average precision over the ranked detections. A detection counts as a true positive only the first time it finds an instance.

10. Results

On-seam artefacts (29 instances, 19 km of seam, out-of-fold)

Scorer	R@1/km	R@2/km	R@5/km	R@10/km	R@20/km	AP
Random ranking	0.03	0.05	0.13	0.23	0.43	0.04
Rule: BD TOPO parallax prior	0.07	0.21	0.34	0.34	0.48	0.07
GBM, context features	0.03	0.17	0.24	0.38	0.59	0.06
GBM, image features	0.14	0.34	0.62	0.86	0.93	0.20
GBM, image + context	0.24	0.41	0.69	0.83	0.90	0.25
SeamNet	0.34	0.55	0.86	0.93	1.00	0.48
SeamNet + GBM (rank average)	0.41	0.59	0.83	0.93	1.00	0.53

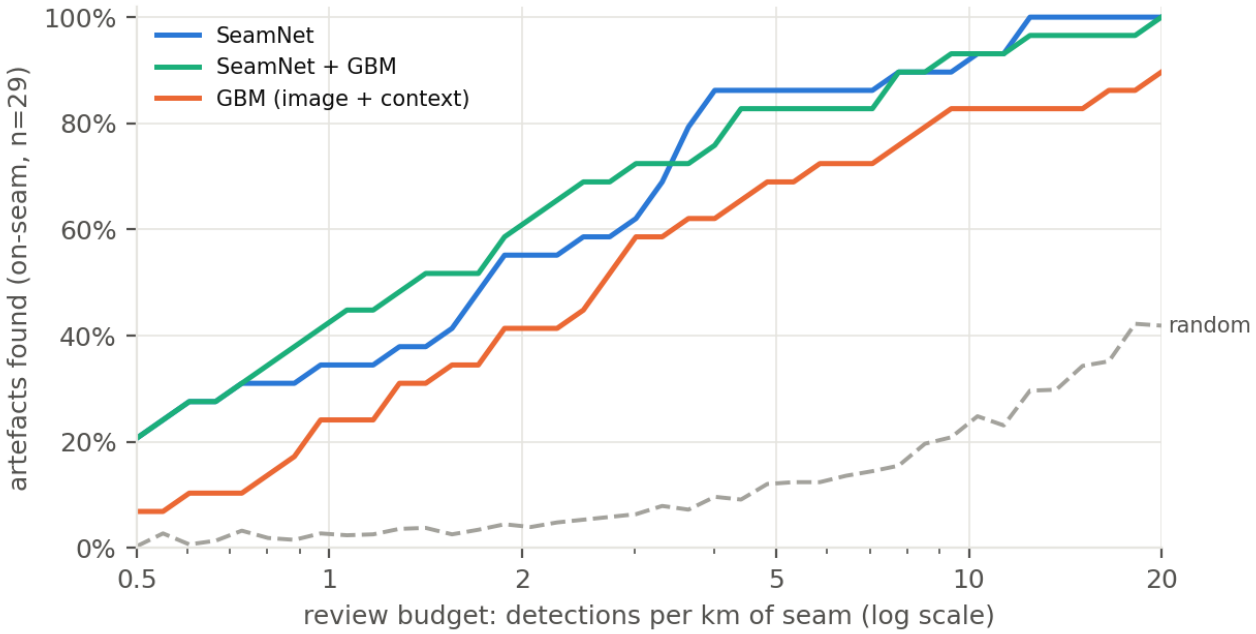


Figure 7. Share of on-seam artefacts found as a function of the review budget. Random ranking is the mean of 20 draws.

All seam-type artefacts in the tiles (34 instances, including 5 points more than 5 m from any seam)

Scorer	R@1/km	R@2/km	R@5/km	R@10/km	R@20/km	AP
Random ranking	0.02	0.04	0.10	0.20	0.38	0.03
Rule: BD TOPO parallax prior	0.06	0.18	0.29	0.29	0.41	0.06
GBM, context features	0.03	0.15	0.21	0.32	0.50	0.05
GBM, image features	0.12	0.29	0.53	0.74	0.79	0.17
GBM, image + context	0.21	0.35	0.59	0.71	0.76	0.21
SeamNet	0.29	0.47	0.74	0.79	0.85	0.41
SeamNet + GBM (rank average)	0.35	0.50	0.71	0.79	0.85	0.45

Per type, at 5 detections per km (on-seam)

Scorer	Bâtiment coupé	Véhicule coupé	Décalage	Erreur de mosaïquage
Rule: BD TOPO parallax prior	10/11	0/15	0/1	0/2
GBM, image + context	10/11	8/15	1/1	1/2
SeamNet	10/11	12/15	1/1	2/2
SeamNet + GBM (rank average)	10/11	11/15	1/1	2/2

Reading the results.

- The BD TOPO parallax rule alone already finds nearly all on-seam bâtiment coupé: when a seam crosses a tall building between two distant photos, a cut is likely. It cannot see vehicles at all.
- SeamNet's gain comes mainly from véhicule coupé, the most frequent label in the full data (898 of the 1,027 vallées points). It needs to *see* the half-vehicle, which hand-made features capture poorly.
- Averaging the SeamNet and GBM ranks helps at small budgets (best at 1–2 per km and best AP). SeamNet alone is best from about 4 per km upwards.
- Uncertainty is large: one artefact is 3.4% of recall. The ranking of the methods is consistent across budgets, but individual values will move with more data.



Figure 8. SeamNet's 12 best detections (out-of-fold, 5 cm crop, seam vertical in the centre). Green: an artefact point lies within 10 m. Orange: no point; these are worth a look, since only artefacts that were sent back are labelled. The caption under each crop is the predicted type.

Two of the three "no point" detections (#5, #6) sit on the same roof with skylights. #10 is a parked car beside the seam. Some of these may be real but unreported artefacts. With the full operator data (including art_renvoi = 0), they could be checked.

How far from fully automatic?

For automatic flagging, what matters is precision: the share of flagged stations that are real errors. The table keeps SeamNet's top detections at several budgets and gives the score threshold that implies. Precision is a *lower bound*, because unreported artefacts count as false alarms.

Top detections per km	Detections	Score threshold	Precision	Recall
0.25	5	0.988	80%	12%
0.5	10	0.959	70%	18%
1	19	0.916	58%	29%
2	38	0.819	53%	47%
5	96	0.627	33%	74%

Top detections per km	Detections	Score threshold	Precision	Recall
10	192	0.477	19%	79%

The most confident detections are mostly real (70–80 % precision at 0.25–0.5 per km), but they cover under a fifth of the artefacts. Today the model is a **triage tool**: it cuts the length of seam an operator has to inspect, while the operator still decides. Automatic flagging becomes realistic only if precision at useful recall rises with more training data. That can be measured with transfer mode once the other campaigns are added (Section 11).

11. Limits and next steps

- **More data, the decisive step.** Add the other two 2025 campaigns to campaigns.yaml and run **train.py --train A B --test C** for each choice of C. Transfer mode trains SeamNet and the GBM on two campaigns, scores the third, and writes recall, AP and the operating points to out/C/transfer_from_A+B.md. It has been tested end to end; crops of campaigns without a cache are read on the fly. No other local data could stand in: the historical tiles in ~/Downloads/CRAIG have lost their seamline geometry (.shp files missing), and the GAM 2025 test seamlines have no matching imagery.
- **Train on GPU.** Use the 4×T4 VM: longer schedules, a larger backbone (ConvNeXt-T), and crops read on the fly. The local AMD GPU fails with a HIP error.
- **Weak negatives.** Only artefacts SG sent back are labelled, so some negatives are unreported artefacts. An export of art_renvoi = 0 and cv_zone_suivi would fix the labels and the precision estimate.
- **Production false-positive rate.** Every tile is centred on an artefact. Before going fully automatic, the false-alarm rate per km must be measured on one complete block (all tiles plus its seamlines).
- **Not built yet.** A 1-D model over consecutive stations (NMS stands in for it), hard-negative mining, and an id for raccord radiométrique (missing from the 2025 export).
- **Hotspots and cloud shadows** are not seam errors; Section 12 covers them. Their rules are fitted on 5 and 3 examples, so they need more labelled data before their numbers mean much.

12. Using it: detect.py, with hotspots and cloud shadows

One command takes tiles (GeoTIFF, Lambert-93, about 5 cm, for example 4000 × 4000 px) and a seamline file (.shp or .gpkg; polygons with photo names, or plain polylines) and writes **errors.gpkg**: one point per error in the cv_artefact schema. It also writes review layers and a per-tile summary. Polylines work because SeamNet's seam channel comes from the drawn line, not from photo names. On the same 33 tiles, the polygon and polyline runs give the same 73 errors.

```
python detect.py --tiles "/data/tiles/*.tif" --seamlines seamlines.shp --out /data/results
                 [--checks seams,hotspots,clouds] [--threshold 0.7] [--no-bdtopo]
```

Check	Method	Eure-et-Loir, 44 tiles
seams	SeamNet probability ≥ threshold at stations every 4 m, after 8 m NMS	out-of-fold at 0.7: ~3.8 errors/km, precision ≥ 41 %, recall 71 %
hotspots	all glints (saturated cores), then the reportable ones: core ≥ 2 m² with a glow beyond its outline (bloom)	5/5 reported, 1 other flag of 797 glints (in-sample)
clouds (experimental)	≥ 80 m of seam where the same open land is darker on one side, same colour, texture scaled like brightness	2/3 found (both cut by a seam), 0 other tiles (in-sample)

Hotspots. In CRAIG's usage (tuto_cv pp. 6, 19–22), a hotspot is a *specular sun glint*: a saturated white disc, often with star rays, on a roof or a windscreen. It is reported only when visible at 1/800 and disturbing. Every reported hotspot in the sample is a saturated blob of 2.4–7.5 m² within 1.4 m of the point. But the tiles hold about 24 glints each, so detection is not the problem: *reportability* is. That explains the earlier YOLO's 3,640 boxes on the GAM test zone. Large white objects (roofs, tents, skylights, pools, a bus) are just as saturated. What separates a glint is **bloom**, light spreading beyond the outline, measured with a distance transform from the core so it works for any shape. The

antisolar-geometry idea was tested and dropped: without exposure positions and times, the offsets point in every direction. With 20 or more labelled hotspots, hotspot.py trains a classifier on the 18 blob features to replace the rule.

Cloud shadows. With 3 examples, the method has to be unsupervised. A shadow belongs to one photo and becomes an artefact where the mosaic exposes it, typically along a seam (one operator's comment: "déplacer la ligne de mosaiquage"). Along each seam, 20 m segments compare the open land 3–25 m on each side, away from BD TOPO buildings and roads. A shadow segment is darker by ≥ 12 % with the same chromaticity, and its texture contrast is reduced in proportion. Only long stretches count: the two shadows cut by a seam span 111 m and 130 m, and nothing else exceeds 51 m. The error point lies on that seam stretch, where the fix applies. Small shadows away from seams (3498, about 20 m across) are missed. Large dark patches go to the review layer only, because alone they do not separate shadows from darker fields.

Appendix: running it

From CRAIG ARTIFACT/seam_qa with the image_analysis/roof venv. In that venv, osgeo.osr segfaults; the code uses pyproj and rasterio only.

```
python detect.py --tiles "tiles/*.tif" --seamlines s.shp --out results   # new data, all checks
python seam_graph.py                  # stations, context, labels, mosaic.vrt
python features.py --workers 8        # cross-seam features
python baseline.py                    # rule + GBM, grouped CV, gbm_all.joblib
python dataset.py                     # two-scale crop cache (small campaigns)
python train.py --steps 250 --final   # SeamNet CV + final model (--device cuda on the VM)
python train.py --train A B --test C --steps 2000 --device cuda   # leave-one-campaign-out
python infer.py --campaign C --models-from model_A+B --budget 10
python hotspot.py --campaign C        # glints, rule; trains the classifier with >= 20 labels
python cloud.py --campaign C          # cloud-shadow candidates and report
python docs/figures.py && python docs/make_report.py
```

Main constants (config.py): STRIDE 4 m; positive radius 3 m; ignore radius 30 m; match radius 10 m; instance merge radius 20 m; NMS radius 8 m; flying height 1,250 m; crops 192 px at 5 and 10 cm.